Recaps

① Week 1:

soft ware arch. :

week 1s

What is Software Architecture?

- it is the Princ. of design decisions.

-> set of struc need a reason about the sys. for: long process & large Proj.

- Documenting is critical

it Provides: => high quality Software.

- Notes - It is an abstraction
- about quality
- it is not a step, it starts from the beginning

② captures how the sys was acting built after the fact

③ Arch. degradation

④ What makes a good arch.?

=> there is no good or bad, arch. are either more fit for specific purpose and context

⑤ • why is S.A Impo.?

• Adaptability, flexability

-> everything changes (Techno, Software, user req. and expectations) change

exs BlackBerry
- OS too old
- No dev support

Nokia Symbian Mobile OS
- Monolithic
- Complex dev. env

Adobe Flash
- security and Performance issues
- Couldn't adapt to fast changes in web

• Increased sys complexity -> scale. need; -> microservices; reuse

- cloud computing and decentralized Infrastruc. -> cloud native arch. -> Container orchestratio

• fast Changing Techno

• user Expect.

• security

Transferable, Reusable Model

⑥ arch. views:

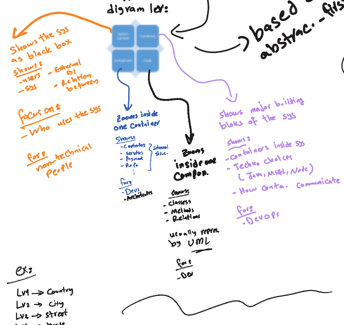
view ① C4 model 2 basic Concept:

abstraction to describe the arch

diagrams to visualize the abstraction

4 supported diagram lev:

-> based on abstrac. - first thinking



ex
Lv1 -> Country
Lv2 -> City
Lv3 -> street
Lv4 -> house

We also have some extras

① Landscape: shows multiple sys related to each other

② Dynamic: shows runtime behavior

③ Deployment: shows how containers are deployed to infra.

③ 4+1 View Model

⑥ view ② Software Arch. in practice views:

Module struc
• Implementation or executable unit
-> the concern: - functionality - Modifiability - Maintainability
• Each module can be assigned to diff team

Component & Connectors (C&C)
- shows runtime elements
- how elc. interact while sys is running

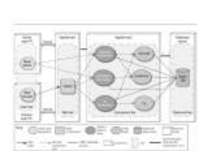
Allocation struc
-> describe mapping between software and env.
Types
- organizational
- development
- Installation
- execution

Notes
Modules = Compile time
Com. & Con. = Runtime
• Modules are built into Components

⑦ Quality view

It is formed by **extracting relevant parts** from one or more of those structures.

Combined view:



Deployment, Communication, Module views

Week 2:

① Quality attr.:

Availability: -> about is sys working?

Fundamentally, availability is about minimizing service outage time by mitigating faults.

-> the goal is make the time sys not working as short as possible (min downtime)

• Operational categories
- Availability
- Interoperability
- Reliability
- Usability
- Performance
- Deployability
- Scalability
- Monitorability
- Mobility
- Compatibility
- Security
- Safety

• Developmental categories
- Modifiability
- Variability
- Supportability
- Testability
- Maintainability
- Portability
- Localizability
- Development distributability
- Buildability

② Basic Properties of Styles

Benefits of styles:

Architectural Patterns

• An architectural pattern is:
- a set of architectural design decisions
- that are applicable to a recurring design problem
- and parameterized to account for different software development contexts in which that problem appears.

What is a Pattern?

Structure of a Pattern

According to the lecture, every architectural pattern defines t

Context
The situation where the problem occurs.
Example:
- building a distributed system
- designing a web application
- handling large data processing.

Problem
The recurring cha
Example:
- how to orga
- how to coord
- how compon

The goal -> what are

Why Architectural Patterns Are Imp

They help achieve system qualities such as:

Quality Attribute	Impact
Scalability	handle gr
Maintainability	easier evo
Reliability	stable stru
Performance	efficient o

These qualities are often the **main reason for selecting a pa**

• Difference between archite software design patterns?

Architectural Patterns vs De

with Notes

Reliability:

→ about if sys is working correctly?

Performance:

→ about how fast the sys is?

Scalability:

→ about when workload is ↑high, but the sys works same. (same resp. time)

1) Design Reuse
You don't reinvent architecture.
You apply a known structure.

2) Code Reuse
Some styles have reusable frameworks:
- Layered frameworks
- Event buses
- Component containers

3) Understandability
If someone says:
"This is a client-server system."
You instantly understand a lot.

4) Interoperability
Standard styles → systems integrate easier.

5) Style-specific analysis
Because structure is constrained:
You can analyze:
- Performance
- Deadlock
- Scalability

6) Visualization
Each style has standard diagrams.
This helps communication.

what is an arch. style?

An architectural style is a named way of organizing a software system.
It gives rules about how parts connect and work together,
and it helps achieve certain quality goals.

⟹ according to these points i categorized each style

① Monolithic Arch.

a system where all parts of the app are buit and deployed as one single unit.

↳ one codebase. one deployment one process. > everything runs together

All functionality is tightly connected and runs together.

There is no separation like client/server or event bus.

Advs
- Easy to:
  - Develop
  - Deploy
  - Test
- simple arch.

disAdvs
- high coupling (everything depends on everything)
- hard to:
  - maintain
  - scale parts independently.
  - modify safely

When Is It Suitable?

ex:

② Layred Arch.

- is a sys divided into sep. layers where each layer has a specific respon. and communicates only with it's neighboring layers.

↳ each layer builds on top of the one below it.

Advs
- ↑ abstraction lev.
- Reuse
- clear separation of concerns
- Easy to: • maintain • modify
- testability
- Standardized layer interface for lib. and framework.

disAdvs
- performance overhead
- May not suit modern infrastructures (not ideal)

Notes
- sometimes real sys need to skip layers

When Is It Suitable?

Layered

③ Client-server

- is a sys where clients req. services and server provides services and manages data over a network.

↳ can be called 2-Tier.

• client and server are sep. and often on diff. machine.

Notes How can we diff between Client-server and any 2-Tier?

→ Client always small, server is big

Advs
- centralized data manag.
- Easier security control
- multiple clients can connect (scalable sys)
- Easier server updates

disAdvs
- server can be bottleneck
- if server crashes → sys down
- network latency effects perf.
- Req. network Ifrastruc.

Client-Server Lunar Lander App

④ Pipe and Filter

- is a style where data flows through a sequ. of independent components (filter), connected by pipes.

↳ each filter - Recives input data
- Processes it
- Produces output data

Advs
- high reuseability
- Easy to add and remove filter
- support parallel exec.
- Good for data transfor.
- Easier analysis

disAdvs
- Not suitable with interactive sys
- Not good for complex shared data
- Bottleneck in one pipe effects whole sys
- Data format constraints

When Is It Suitable?
- Compilers
- Data processing systems
- ETL pipelines
- Streaming systems
- Log processing
- Signal processing

Important Exam Idea

Variations:

Types: • Types of Filters

Producer — Consumer — OR

⑤ Component-Based Arch

- is a style where a sys is buit from independent reusable components that communicate through interfaces.

obj. req. broker

Advs
- high reusabili.
- faster deve.
- Independent dev team
- replace compon. without rebuilding entire sys
- reliability

disAdvs
- Integration complex.
- Compatibility issues
- Perfor. overhead
- version manage

⑥ Hexagonal (Ports & Adapters)

- is a style where the core buss. logic is placed in the center, and all external sys (UI, db, API, ...) connect to it through ports and adapters.

Main Idea

External systems depend on the core.

⑦ Plugin or microkernel

- is a style where a small core sys provides basic func., and additional features are add as independent plugins.

↳ e.g. Browsers, some apps "notion, obsidian, ..."

Important Principles
- Independent Plugin Principle
- Standard Interface Principle
- Loading Strategies

Advs
- Extensibility
- maintainability
- Scalability
- flex.
- Easy feature addition

disAdvs
- performans overhead
- Compatibility
- Security → 3rd Party apps
- Complex than mono.

⑧ Blackboard (Repo)

- is a style where multiple independent components communicate by reading and writing to a shared data struc., called Blackboard.

↳ the blackboard controls the sys progress

Important Characteristics
- Central data structure
- Shared state
- Components triggered by data changes
- Often used in AI systems

Advs
- General style suitable for network-based apps
- suitable for DBs
- good for collaborative

disAdvs
- may become bottleneck
- Complexity (data integrity)
- sync challenges

---

① layered :

- state - logic - Display
- Model - view - Controller MVC

Definition

Layered patterns are architectural patterns where the system is divided in... each layer has a specific responsibility.

The idea is to separate parts of the system that change at different ra...

Your slide says:

"separate elements with different rate of change"

Meaning:

Some parts of a system change often (like UI), while others change rarely ... Layered patterns separate these parts to make the system easier to ma...

1. State - logic ...

→ separetes the sys into

- state - logic

Topology (Structure)

Components:

State
Stores the system data.
Examples:
- database
- application state
- stored information

Logic
Contains the rules and processing of the sy...
Examples:
- calculations
- validation
- business rules

Main Idea
Different parts of software change at different speeds.

Advantages

Suitable Applications
This pattern is useful in systems where:
- UI changes frequently
- logic should remain stable
- data must be protected

Examples:
- desktop applications
- early GUI systems
- interactive software

Additional notes

State-Logic-Display (a.k.a. Three-Tiered Pattern)

State-Logic-Display (a.k.a. Three-Tiered Pattern)

Definition
Component pattern focus on how independent components or services comm... interact with each other.

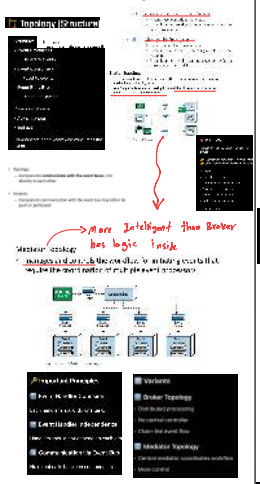
These patterns are especially important in:
- distributed systems
- microservices
- service-based architectures

Enterprise Java Beans

- Sun Microsystems: 1990

⑨ Event-Driven

- is a style where Components communicate by producing and consuming events async through an event bus.

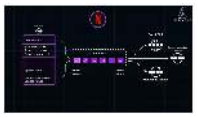


Adv's

- high scalability
- loose coupling
- Easy to extend
- good for distributed sys
- support async
- High levels.

disadv's

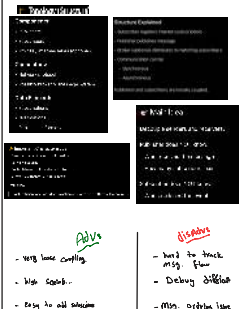
- hard to debug
- Hard to trace exec. flow
- Complex event coordination
- Event ordering problem
- Eventual Consistency issues



⑩ Publish-Subscribe

- is a pattern where publishers send messages without knowing who receives them and subscribers receive messages without knowing who sent them.

↳ It is a decoupled form of event-driven arch



Adv's

- very loose coupling
- high scalab.
- easy to add subscribers
- Flexible
- good for distributed sys
- Efficient event broadcasting

disadv's

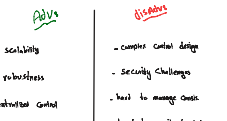
- hard to track msg. flow
- Debug difficult
- msg. ordering issue
- Event delivery guarantees can be complex
- Performance bottleneck in broker

Twitter · RSS Feeds · Email

⑪ Peer-to-Peer (P2P)

- is a style where all nodes are equal and each node can act as both Client and Server.

↳ No central authority controlling the system

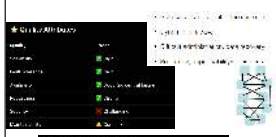


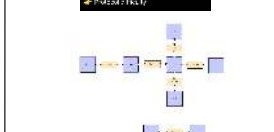
Adv's

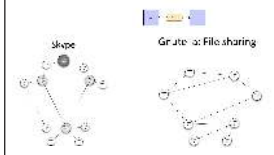
- High scalability
- High robustness
- Decentralized Control
- Efficient resource sharing
- Good fault tolerance

disadv's

- Complex Control design
- Security challenges
- hard to manage consis.
- hard to monitor/control
- Debugging is diff.



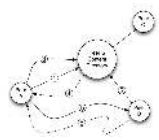




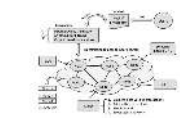
Skype

Gnutella: File sharing

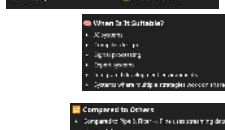
Napster: Music sharing "Application"

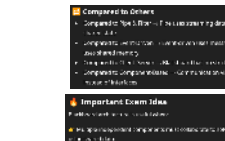


The Bitcoin peer to peer network

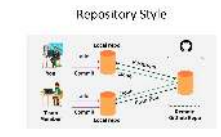


Week 3:

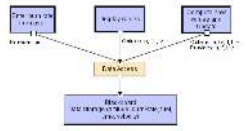




Database · Tuple space · Expert systems (AI)

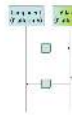


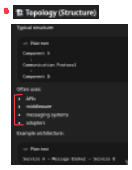
Repository Style

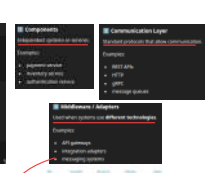


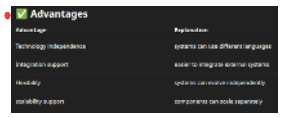
1- Interoperability

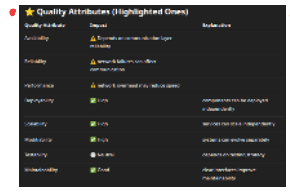
→ It enables communication between technologies, platforms, or proc





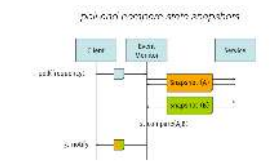


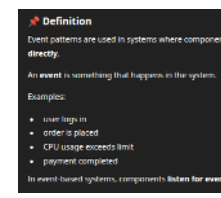




1. Event monitor part.

- Notifies client when a specific event occurs in a service.





Distributed System

- A system with multiple components located on *different machines* that communicate and coordinate actions in order to *appear as a single coherent system* to the end-user.

What are the benefits?

- Improved *reliability*
- Improved *scalability*
- Improved *latency*

What are the drawbacks?

- Increased *complexity*
- Increased *attack surface*
- Increased *latency*
- Introduce *consistency* problems

① Service Oriented Arch

S.O Arch is the composition of software components (services) to form a complete sys.

Topology (Structure)

Advs

- loose coupling between services
- Reusability of services.
- Good Interoperability
- Integrates legacy sys.
- suitable for large enterprise sys.
- standard connectors & interfaces

disadvs

- ESB problems: - central bottleneck - hard to scale - heavy governance - complex deploy - Business logic moving into ESB
- Other issues:
- Shared DB makes services hard to manage.
- failure of one services may effect workflow.

Web Services (WS-*) · Cloud Computing

Additional points from the slides (service-oriented Architecture)

Basic Topology of Service-Based Architecture

Enterprise Service Bus (ESB)

Why ESB Became a Problem

API Layer (Modern Replacement)

Interface Compatibility Problem

② REST

- is an arch. style for distributed sys defining certain constraints that provide efficiency, scal. and other desirable attr.

Topology (Structure)

REST Constraints (Very Important in Slides)

Other Important REST Concepts (From Slides)

Advs

- scalable
- loose coupling
- Interoperability
- Performance Improv. via caching
- simplicity

disadvs

- Statelessness can increase work.
- Not ideal for complex Transactions.
- over-fetching or under-fetch.
- Network overhead

Quality Attributes

Suitable Applications

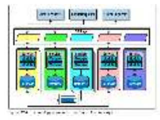
Compared to Other Styles

Important Exam Idea

World Wide Web · RESTful Web APIs

③ Microservices Arch.

- is an evolution of Service-oriented Arch (SOA)



Important Principles (From Slides)

Note:

- Microservices are NOT modules or functions
- Avoid synchronous logical flow

Composition of Microservices

- How to compose microservices?

Advs

- high modularity
- high extensibility
- high scala.
- high Interoperability
- Improved reliability

micro serv. also good for: Cloud env. / Continuous deployment / Indepen. Scaling

disadvs

micro sev. are not good for: simplicity / testability

Additional challenges Includes:

- distributed sys complexity
- network latency
- Deployment difficulty

Quality Attributes

Suitable Applications

Compared to Other Styles

Important Exam Idea

④ Distributed Monolith (Anti-Pattern)

- the essence of the distributed mono. is lack of isolation between services.

Topology (Structure)

Main Idea

Key Traits (From Slides)

it may have one or 2 adv when it do migration but still a very bad style

Why It Is an Anti-Pattern

Quality Attributes

When Does This Happen?

Compared to other styles

Important Exam Idea

⑤ Migrating a monolith

- it decompose the monolith into services using bounded contexts and Indep. services.

Topology (Structure)

Key Principles

Important

Strangler Fig Pattern

those are the steps of the St. Fig patt in detail

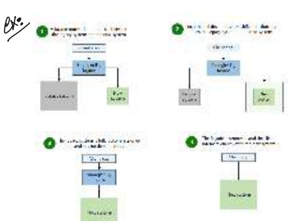
Advs

- Gradual sys modernization
- Reduce migration risks
- Continuous sys oper.
- Better scala.
- Improved modularity

disadv.

- migration Complexity
- Temporary hybrid arch.
- Increased sys Complexity during transitions.
- Need for careful Service boundaries

Ex:



Ex:

Slayts last part

Fallacies of Distributed Computing

The Network Is Reliable

Network Performance Is Always Good

Bandwidth Is Unlimited

Security Is Not a Problem = Network is Secure

Network Topology Does Not Change

There is only one administrator

- Too many layers in current applications
  - Internet Service Provider; Cloud provider; Client Data Centers, etc.

Distributed Systems Are Cheap

Transport cost is zero

Definition

Composition patterns describe **how multiple services** ... **perform a task**.

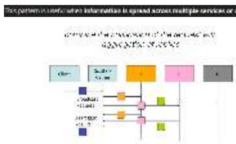
These patterns are especially used in:

- distributed systems
- microservices architectures
- large-scale data processing systems

1- Scatter - Gather

- Distributes a req. to multiple services in parallel, collects their res., and then combines the res. into one final res.

This pattern is useful when information is spread across multiple services or data sources.



Topology (Structure)

Main Idea

Example

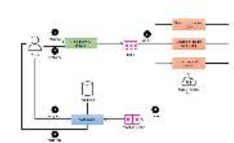
Advantages

Disadvantages

Quality Attributes (Highlighted Ones)

Suitable Applications

Real-World Example



2 - lo...

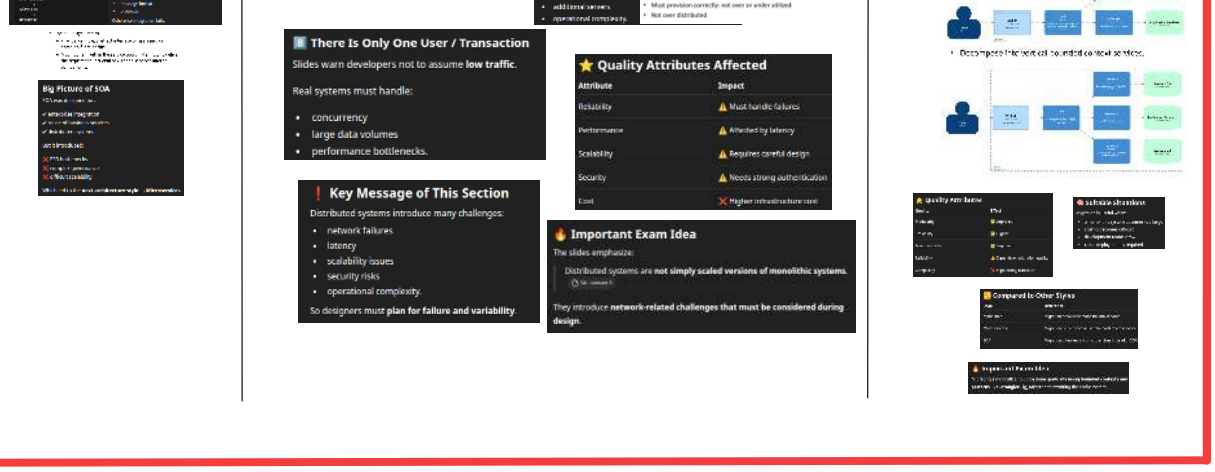
Topology Structure

Main Idea

Example

Advantages

Quality Attributes



Archit

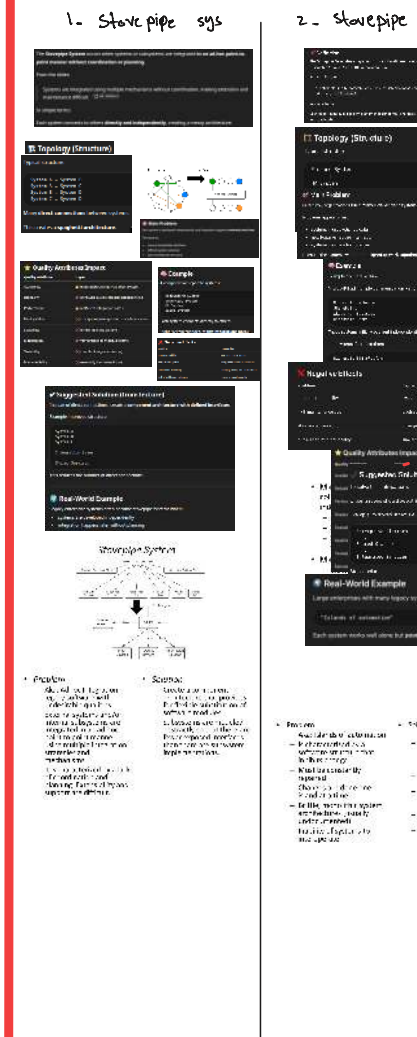
Definition

An **anti-pattern** is a **commonly used solution that ac**
them.

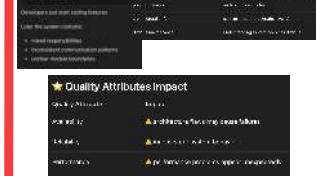
From the lecture:

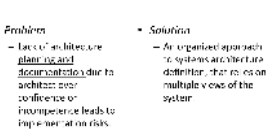
> An Anti-Pattern describes a commonly occurring so
> consequences. SA - Lecture 4

So unlike patterns (good solutions), anti-patterns are **b**

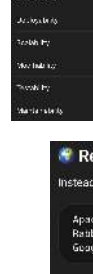


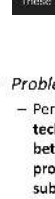
5 - Arch. by Implication





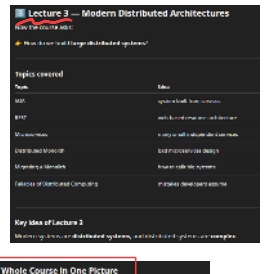
- *Problem*
  - Lack of architecture planning and documentation due to architect over confidence or incompetence leads to implementation risks
- *Solution*
  - An organized approach to system architecture definition, that focus on multiple views of the system



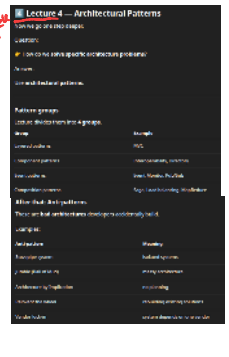


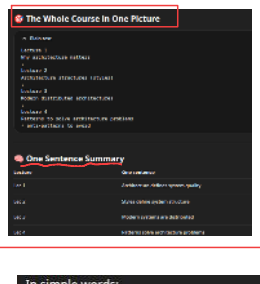
- *Proble*
  - Per
    tec
    bet
    pro
    sub
  - *Qu*
  - *Cor*
    not
  - *No*
    pro

Week 4:



this is the Lec. week 4 Topic

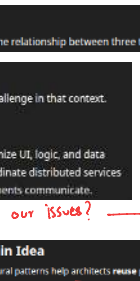




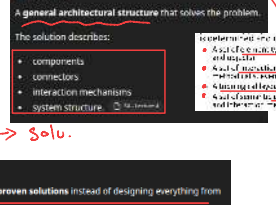
In simple words:

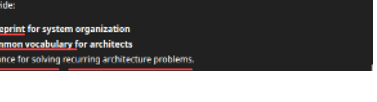
👉 An architectural pattern is a **general reusable solution for a common architecture problem.**

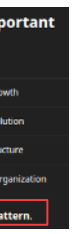
It is not a finished design, but a **template** that architects can adapt to different systems.

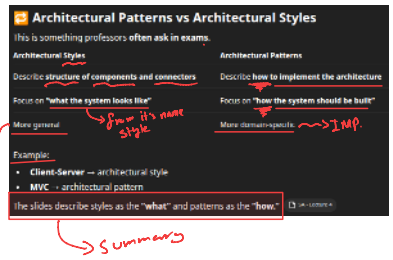


our issues? → solu.





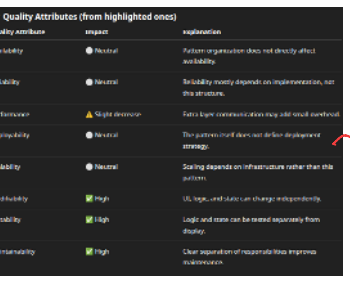


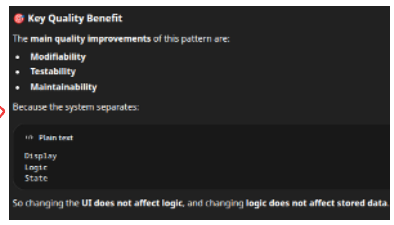


ectural patterns and

Sum.

	Architectural Patterns	Software Design Patterns
Definition	High-level structural solutions that define the overall system organization	Reusable solutions to common problems in software design at the code level
Scope	Large-scale (entire system or subsystems)	Small-scale (class or object level)
Focus	System structure, component relationships, communication	Object interaction, class structure, responsibilities
Goal	Ensure scalability, performance, reliability, and maintainability	Improve code reusability, flexibility, and maintainability
Impact of Change	Affects the entire application structure	Affects specific classes or modules





🔥 Important Exam Idea

Architectural patterns provide:

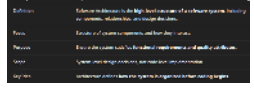
> **Reusable high-level solutions for recurring architectural problems.**

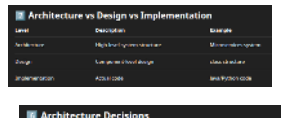
They operate at the **system architecture level**, not the code level.

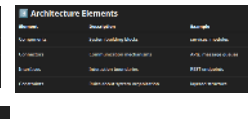
The arch. Pat. are 4 groups:

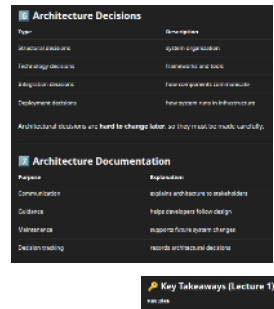
Summary of Lec 1,2,3,

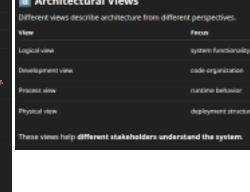
Lecture 1 — Software Architecture Fundamentals

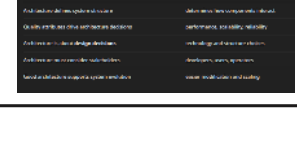




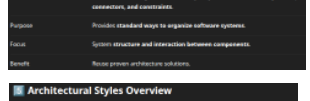


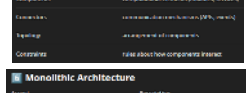


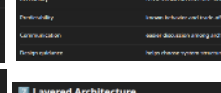


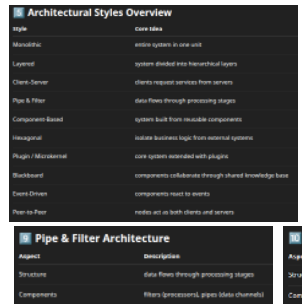


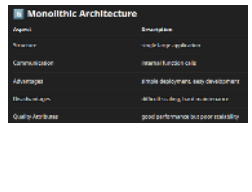
Lecture 2 — Architectural Styles

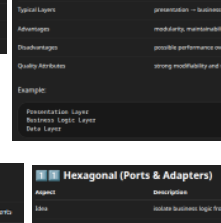


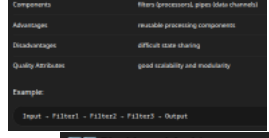


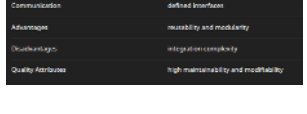


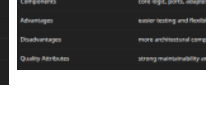


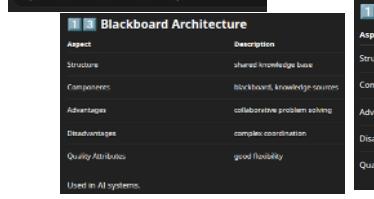


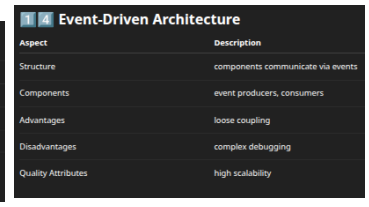


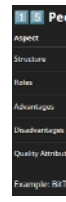


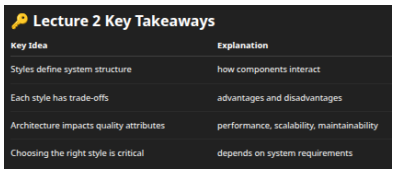




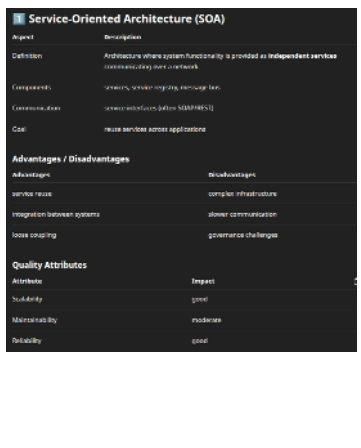


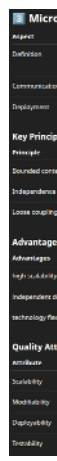






Lecture 3 — Distributed Systems & Service Architectures





② Component :

- Interoperability
- Directory pattern :

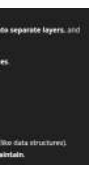
③ events :

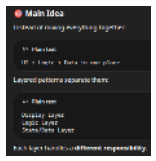
Notification Patterns:
- Event Monitor
- Publish - Subscribe
- Messaging bridge
- Half Sync / Half Async

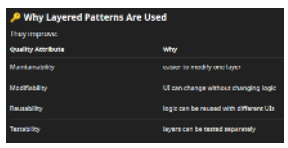
④ Composition:

- Scatter - Gather
- Load Balancing
- Orchestration
- Choreography
- Saga pattern
- Map - Reduce

⑤ layered :



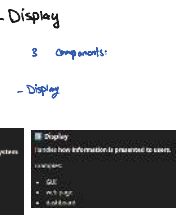


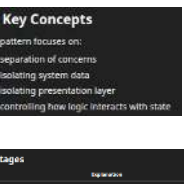


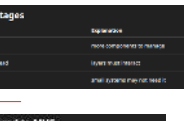
- Display

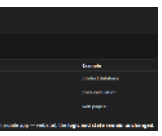
3 components:

- Display





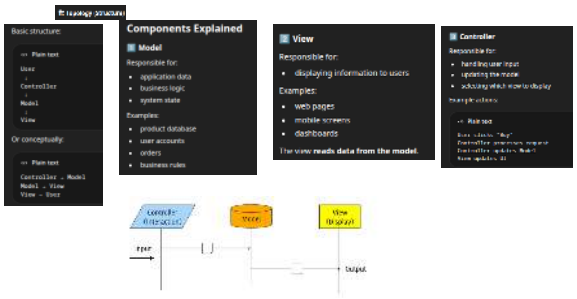


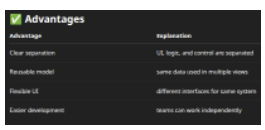


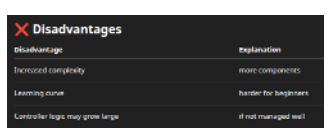
2. Model - view - Controller

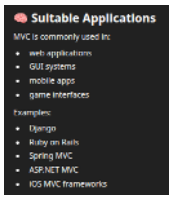
→ Separates an app into 3 components:

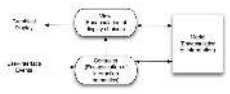
- Model
- View
- Controller

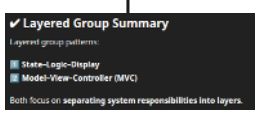




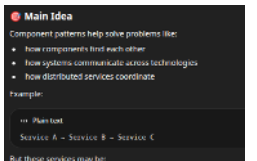


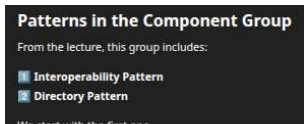


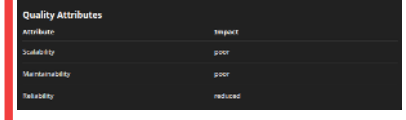


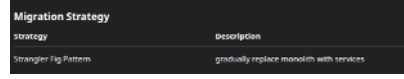


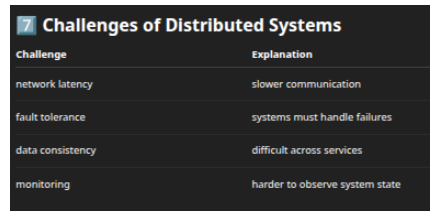
② Component :

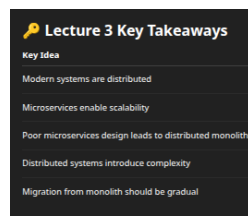


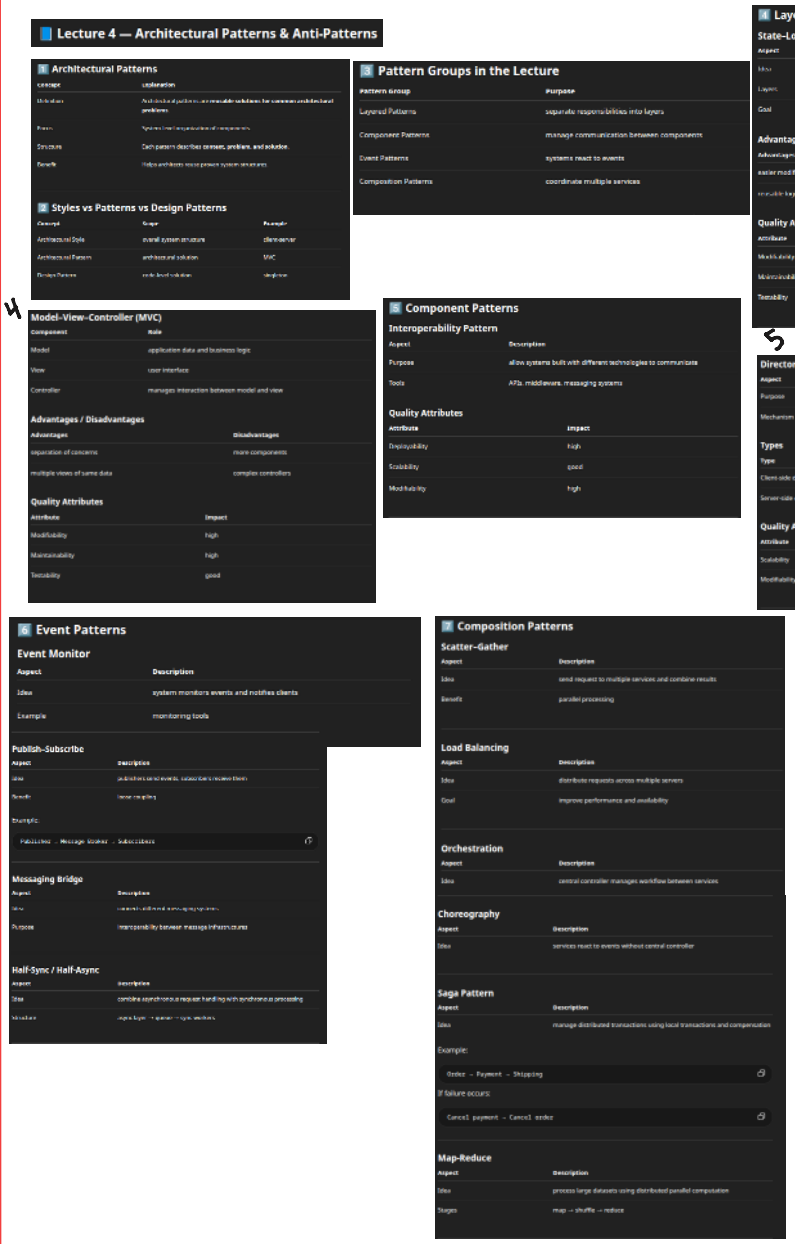




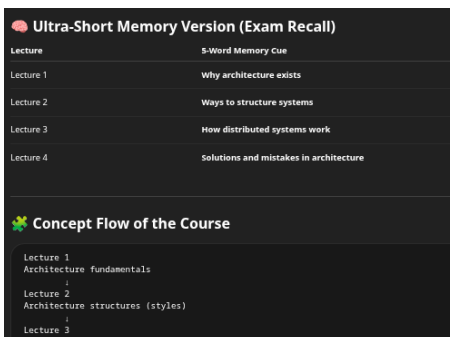








Summary of Sum.



- written in different languages
- running on different servers
- using different technologies

Component patterns **enable interoperability between them.**

We start with the first one.

2- Directory

📌 Definition

The **Directory Pattern** allows components or services in a distributed system to **find each other dynamically** without knowing their exact location.

From the lecture:

> It provides **location transparency**, meaning components can discover services without knowing their physical address.

In simple words:

Instead of hardcoding service addresses, systems **ask a central directory where the service is located.**

🏗 Topology (Structure)

Typical structure:

```
Client
Service Registry (Directory)
Service Instance
```

Example flow:

1. Service registers itself in the **directory**
2. Client queries the **directory**
3. Client receives the **service address**
4. Client communicates with the service

1️⃣ Service Registry (Directory)

A central registry storing:

- service names
- service locations
- service health status

Examples:

- DNS
- service registry
- discovery server

2️⃣ Client

The system requesting a service.

Example:

- web application
- microservice
- API gateway

3️⃣ Service Instance

The actual service providing functionality.

Example:

- payment service
- authentication service
- product service

🔀 Two Types of Directory Pattern

1️⃣ Client-Side Discovery

The client directly queries the service registry.

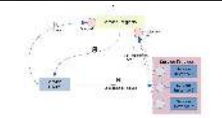
```
Client → Registry → Service
```

Process:

1. Client asks registry for service location
2. Registry returns service instance
3. Client communicates directly with the service

Example technologies:

- Netflix Eureka
- Consul



2️⃣ Server-Side Discovery

The client sends requests to a **load balancer or gateway**, which handles discovery.

```
Client → Load Balancer → Service
```

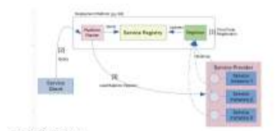
query

Process:

1. Client contacts load balancer
2. Load balancer queries registry
3. Load balancer forwards request

Example technologies:

- Kubernetes service discovery
- AWS load balancer



Server-Side Discovery

- Client sends request to a fixed endpoint, usually a load balancer or API gateway.
- The load balancer queries the service registry to pick a healthy service instance.
- Load balancer forwards the request to that service instance.
- Client only interacts with the load balancer.

🎯 Main Idea

The directory pattern removes **hardcoded service locations.**

Instead of:

```
Client → Service IP address
```

You have:

```
Client → Directory → Service
```

This allows services to **move or scale without breaking the system.**

✅ Advantages

Advantage	Explanation
Location transparency	clients don't need to know service location
Dynamic service discovery	services can move or scale
Flexibility	easier service management
Improved scalability	services can be replicated

❌ Disadvantages

Disadvantage	Explanation
extra infrastructure	registry server required
possible single point of failure	if registry fails
additional network communication	lookup overhead

⭐ Quality Attributes (Highlighted Ones)

Quality Attribute	Impact	Explanation
Availability	⚠ Depends on registry availability	
Reliability	⚠ registry failure affects service discovery	
Performance	⚠ lookup adds small latency	
Deployability	✅ High	services can be deployed dynamically
Scalability	✅ High	new services can register automatically
Modifiability	✅ High	services can change location without affecting clients
Testability	⚪ Neutral	testing depends on system design
Maintainability	✅ Good	centralized service management

🧠 Suitable Applications

Directory pattern is widely used in:

- microservices architecture
- cloud systems
- distributed applications
- service-oriented architectures

🌍 Real-World Example

Example: **DNS (Domain Name System)**

Instead of using server IP:

```
142.250.74.14
```

We use:

```
google.com
```

DNS acts as a **directory that maps names to locations.**

✔ Component Group Summary

Component patterns studied:

1️⃣ **Interoperability Pattern**
2️⃣ **Directory Pattern**

These patterns help systems **communicate and discover services** in distributed environments.

③ events :

Notification Patterns:

...nts react to events instead of calling each other

...nts and react when they occur.

🎯 Main Idea

Instead of direct communication:

```
Service A → Service B
```

Event systems work like:

```
Event happens → Components react
```

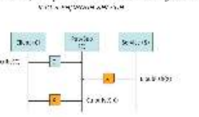
Example:

```
Order Placed Event
    ↓
Payment Service
Shipping Service
Notification Service
```

Each component **subscribes to events and responds when needed.**

2- Publish - subscribe

an event-based arch where publishers msgs (event) and subs. recieve them, knowing each other directly.



3- Messaging Bridge

- is used to connect two diff. msging sys. or msg. buses that are not directly compatible.

In simple words:

If two systems use **different message protocols**, the bridge converts and transfers messages between them.

2- Half-Sync / half-Async

- seperates a sys to 2 parts

an async that handles Incoming req.

a sync that processes tasks sequentialy

Add a layer hiding asynchronous interactions behind a synchronous interface

```
Lecture 4
Patterns to solve problems
+ anti-patterns to avoid
```

📚 Software Architecture — Key Differences Between Lectures (1–4)

Lecture	Core Focus	Main Question Answered	Key Concepts	Architecture Level	Example Topics	One...
Lecture 1	Fundamentals of Software Architecture	*Why architecture matters and what it consists of*	components, connectors, quality attributes, stakeholders, architectural decisions	Conceptual foundation	quality attributes, architectural views, trade-offs	**Arch... dete... like**
Lecture 2	Architectural Styles	*What structural patterns can organize a system*	components, connectors, topology, constraints	Structural system design	layered, client-server, pipe-filter, event-driven, P2P	**Arch... orga... spec...**
Lecture 3	Distributed Architectures	*How modern large systems are built and deployed*	services, distributed systems, microservices, REST, system migration	System architecture evolution	SOA, REST, microservices, distributed monolith, migration patterns	**Mod... serv... desi...**
Lecture 4	Architectural Patterns & Anti-Patterns	*How to solve architectural problems and avoid bad designs*	reusable patterns, event systems, service coordination, architecture mistakes	Architectural problem-solving	MVC, Saga, MapReduce, orchestration, anti-patterns	**Arch... syst... arch...**

🎯 Software Architecture — Professor Trap Sheet

1️⃣ Definitions (Very Common Questions)

Question	Expected Answer
What is Software Architecture?	**The high-level structure of a software system**, including components, connectors, and architectural decisions that achieve system quality attributes.
What is an Architectural Style?	**A family of systems defined by components, connectors, topology, and constraints** that determine how systems are organized.
What is an Architectural Pattern?	**A reusable solution to a recurring architectural problem** in a specific context.
Difference between architectural style and pattern	Style defines **system structure**, pattern provides **solution to architectural problem.**
Difference between architectural pattern and design pattern	Architectural patterns apply to **system architecture**, design patterns apply to **class/code level design.**

2️⃣ Quality Attributes (Almost Always Asked)

Attribute	Meaning
Availability	system operational uptime
Reliability	system works without failure
Performance	response time and throughput
Scalability	ability to handle increased workload
Deployability	ease of deploying system
Modifiability	ease of changing functionality
Testability	ease of testing system components
Maintainability	ease of maintaining system

3️⃣ Styles I...

Style	
Monolithic	
Layered	
Client-Server	
Pipe & Filter	
Component-Based	
Event-Driven	
Peer-to-Peer	

4️⃣ Microse...

Question	
What is Microservic...	
Main principle	
Main advantages	
Main disadvantage...	

5️⃣ REST Questions

Question	Answer
What is REST?	architectural style for **stateless web services using HTTP.**
REST operations	GET, POST, PUT, DELETE
REST principle	stateless communication

6️⃣ Distributed System Questions

Question	Answer
What is a distributed monolith?	system appears distributed but **services are tightly coupled** like a monolith.
Why distributed systems are complex	network latency, failures, consistency challenges
Migration pattern from monolith	**Strangler Fig pattern**

7️⃣ Pattern Recognition Qu...

Pattern	Key...
MVC	sepa...
Directory Pattern	serv...
Publish–Subscribe	pub...
Scatter–Gather	distr...
Load Balancing	distr...
Orchestration	cent...
Choreography	serv...
Saga	distr...
MapReduce	distr...

7

Monolith vs Microservices

Feature	Monolith	Microservices
Deployment	single unit	independent services
Scalability	limited	high
Complexity	low	high
Maintenance	difficult as system grows	easier modular updates

Client-Server vs Peer-to-Peer

Feature	Client-Server	Peer-to-Peer
Control	centralized server	decentralized
Scalability	moderate	high
Fault tolerance	server failure critical	more resilient

8️⃣ Comparison Questions

Orchestration vs Choreography

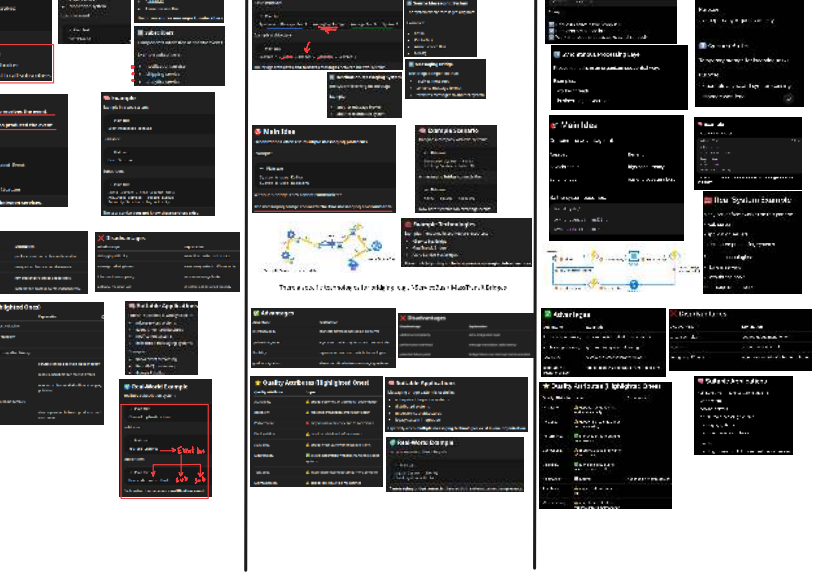
Feature	Orchestratio...
Control	centralized
Workflow	controlled by...
Scalability	moderate
Monitoring	easier

9️⃣ Anti-Patterns (Very Common Short Questions)

Anti-pattern	Meaning
Stovepipe System	ad hoc integrations between systems
Stovepipe Enterprise	isolated enterprise systems
Autogenerated Stovepipe	legacy monolith wrapped as services
Ball of Mud	chaotic architecture
Architecture by Implication	architecture not explicitly defined
Reinvent the Wheel	building existing solutions
Vendor Lock-in	dependence on specific vendor technology

🔟 Pattern Groups (Lecture 4)

Group	Purpose	Patterns
Layered	separate responsibilities	State-Logic-Display, MVC
Component	communication between components	Interoperability, Directory
Event	event-based communication	Event Monitor, Pub/Sub, Messaging Bridge
Composition	coordinate multiple services	Scatter-Gather, Load Balancing, Saga



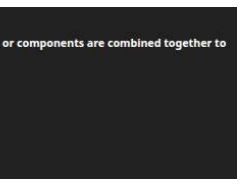
Event Pattern Group Summary

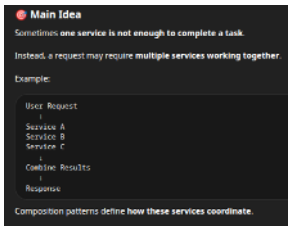
Event patterns studied:

1. Event Monitor
2. Publish–Subscribe
3. Messaging Bridge
4. Half-Sync / Half-Async

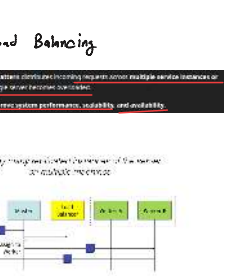
These patterns focus on event-based communication and asynchronous processing.

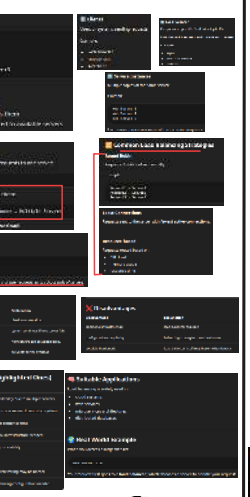
(4) Composition:

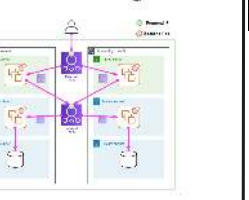




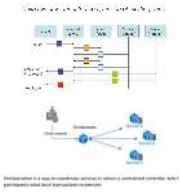
Load Balancing

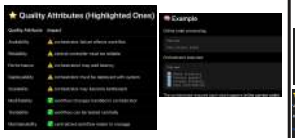






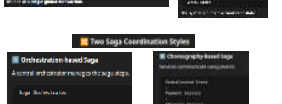
3 - Orchestration

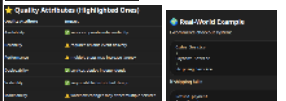


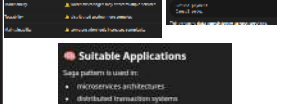


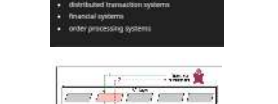
4 - Choreography

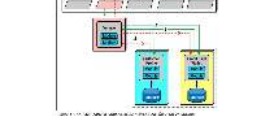
5- Saga Pattern



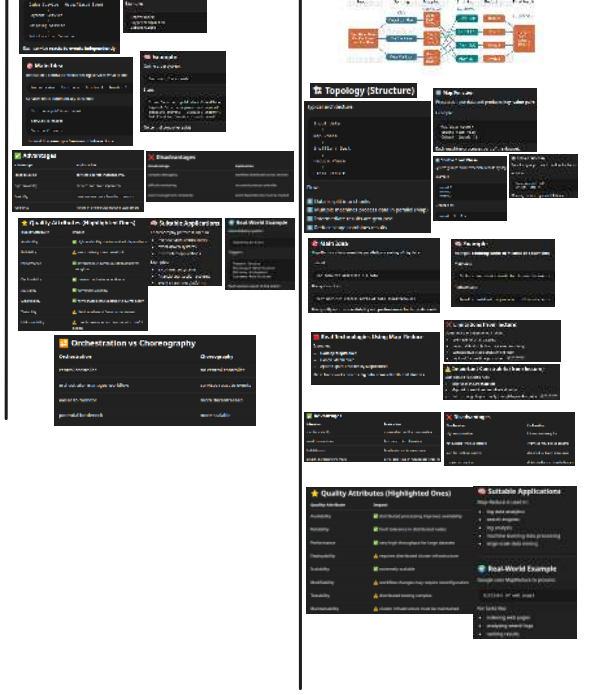








6- Map - Reduce Pat.



ectural Anti-Patterns

tually causes problems instead of solving

lution to a problem that generates negative

ad architecture practices.

🎯 Why Anti-Patterns Happen

Anti-patterns usually occur when:

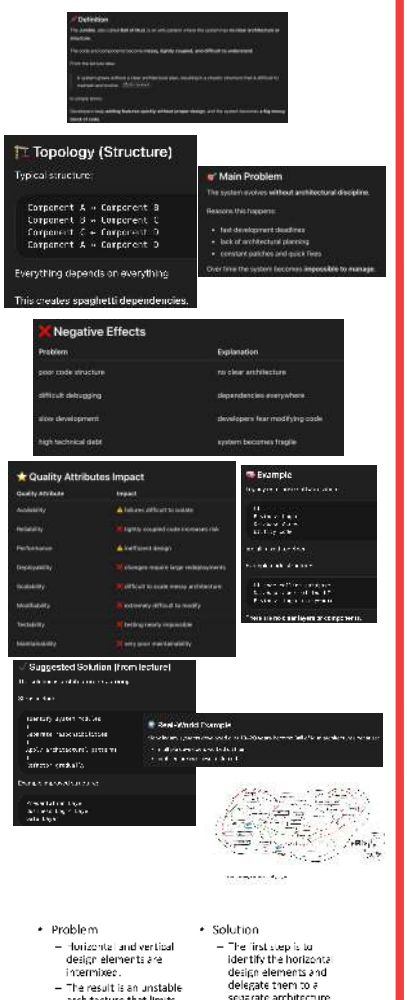
- architects lack experience
- good patterns are used in the wrong context
- architecture is not planned properly
- systems evolve without structure

1- **Stovepipe System**
2- **Stovepipe Enterprise**
3- **Autogenerated Stovepipe**
4- **Jumble (Ball of Mud)**
5- **Architecture by Implication**
6- **Reinvent the Wheel**
7- **Vendor Lock-in** SA - Lecture 4

Enterprise

3_ Auto generated stovepipe

4_ Jumple (Ball of mud)



- Problem
  - Horizontal and vertical design elements are intermixed.
  - The result is an unstable
- Solution
  - The first step is to identify the horizontal design elements and delegate them to a separate architecture

– The layered pattern is violated.

– Then use the horizontal elements to capture the common interoperability functionality in the architecture.

6- Reinvent the wheel

Typical Situation

Suggested Solution (from lecture)

Reinvent the Wheel

Attributes Impact

Real-World Example

...d of building a custom distributed messaging system, companies use:

```
...che Kafka
...bitMQ
...gle Pub/Sub
```

...systems are already **tested, scalable, and reliable.**

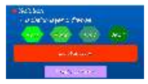
- ...em
  - ...vasive lack of technology transfer between software projects leads to substantial reinvention.
  - *...r problem is unique*
  - *...porate knowledge is ... leveraged*
  - *...explicit architecture ...cess*
- *Solution*
  - Design knowledge buried in legacy assets can be leveraged to reduce time-to-market, cost, and risk.
  - Many ready to use products that are well tested.

7- Vendor lock-in

Definition

Main Problem

- *Symptoms*
  - *"Our architecture is CORBA, Microsoft, and Oracle"*
  - *"We don't have an architecture."*
  - *"We completely dependent on vendor X."*
  - *" Vendor X is the best in the market."*



Example

A company builds a system using many AWS-specific services

❌ Negative Effects

Problem	Explanation
limited flexibility	system tied to one vendor
high migration cost	switching platforms very expensive
reduced negotiation power	vendor controls prices
technology dependency	system constrained by vendor ecosystem

⭐ Quality Attributes Impact

Quality Attribute	Impact
Availability	⚠️ dependence on vendor infrastructure
Reliability	⚠️ failures in vendor platform affect system
Performance	⚠️ limited optimization options
Deployability	❌ deployment limited to vendor environment
Scalability	⚠️ tied to vendor capabilities
Modifiability	❌ architecture constrained by vendor APIs
Testability	⚠️ vendor-specific testing environments required
Maintainability	⚠️ system maintenance tied to vendor technology

Real-World Example

Suggested Solution (from lecture)

To avoid vendor lock-in:

```
Use open standards
Use portable technologies
Design vendor-independent architectures
```

Example approach:

Week 5

4

Stakeholders in Software Architecture

Stakeholder	Interest
Developers	system structure and implementation
Architects	design decisions and trade-offs
Project managers	cost and timeline
Users	usability and performance
Operators	deployment and maintenance

Architecture Trade-offs

Architects must balance competing qualities.

Example:

Trade-off	Example
Performance vs Maintainability	optimized code harder to modify
Security vs Usability	strong security may reduce usability
Scalability vs Complexity	distributed systems increase complexity

Architecture often involves **choosing the best compromise.**

Architectural Styles and Quality Attributes

Quality Attribute	Impact of Architecture
Performance	depends on communication overhead
Scalability	distributed architectures scale better
Reliability	redundant systems increase reliability
Modifiability	modular architecture supports change
Maintainability	well-structured systems ease maintenance

Client–Server Architecture

Aspect	Description
Structure	clients request services from server
Components	clients, server
Advantages	centralized control
Disadvantages	server bottleneck
Quality Attributes	good maintainability but limited scalability

Example:

```
Client → Server → Database
```

Plugin / Microkernel Architecture

Aspect	Description
Structure	main system + plugins
Core	basic system functionality
Plugins	extend system capabilities
Advantages	extensibility
Disadvantages	plugin management complexity
Quality Attributes	high modifiability

Example: IDE plugins

er-to-Peer Architecture

	Description
	decentralized nodes
	each node acts as client and server
	high scalability and resilience
	security and coordination challenges
es	strong scalability

...rrent...

oservices Architecture

	Description
	System composed of **small independent services**, each responsible for a specific business capability.
n	REST APIs or messaging
	independent deployment for each service

les

	Meaning
od	service represents a specific domain
	services developed and deployed separately
	minimal dependency between services

s / Disadvantages

	Disadvantages
	distributed complexity
eployment	difficult debugging
ility	network overhead

ributes

	Impact
	very high
	high
	very high
	difficult

Fallacies of Distributed Computing

Common **false assumptions** when designing distributed systems.

Fallacy	Reality
Network is reliable	networks fail
Latency is zero	network delays exist
Bandwidth is infinite	bandwidth limited
Network is secure	security risks exist
Topology doesn't change	infrastructure evolves
One administrator	multiple control domains

Transport cost is zero	network communication expensive

	Explanation
	services communicate over networks
	independent service architecture
s	architecture must be planned carefully
	networking, failures, coordination
	use patterns like Strangler Fig

...ered Patterns

...gic-Display Pattern

	Description
	separate system into **state, logic, and display**
	display → logic → state
	separate parts that change at different rates

...ges / Disadvantages

	Disadvantages
...fication	additional complexity
...c	communication overhead

...ttributes

	Impact
	high
...ity	high
	high

...y Pattern

	Description
	services dynamically discover each other
	service registry or directory

	Description
...discovery	client queries registry
...discovery	load balancer performs lookup

...ttributes

	Impact
	high
...y	good

Architectural Anti-Patterns

Anti-patterns describe **bad architecture practices**.

Anti-Pattern	Problem
Stovepipe System	uncoordinated point-to-point integrations
Stovepipe Enterprise	isolated systems across organization
Stovepipe	legacy monolith wrapped as services
Jumble (Ball of Mud)	chaotic system structure
Architecture by Implication	no planned architecture
Reinvent the Wheel	rebuilding existing solutions
Vendor Lock-In	strong dependency on specific vendor

Impact of Anti-Patterns

Quality Attribute	Effect
Maintainability	severely reduced
Modifiability	difficult
Scalability	limited
Reliability	unstable systems

...e Exam

...s like:

	Which Lecture
	Lecture 1
	Lecture 2
	Lecture 3
	Lecture 4

-Sentence Summary

itecture defines the high-level structure of a system and ermines how design decisions achieve quality attributes performance, scalability, and maintainability.

itectural styles define common structural ways to anize system components and communication to achieve cific system qualities.

dern software systems are distributed collections of ices that communicate over networks, requiring careful gn to manage scalability, reliability, and complexity.

itectural patterns provide reusable solutions for common em design problems, while anti-patterns describe common itectural mistakes to avoid.

dentification Questions

	Key idea	Typical example
	single large application	traditional enterprise apps
	system divided into layers	web apps
	clients request server services	web systems
	data processed in stages	compilers
	reusable components	enterprise software
	systems react to events	messaging systems
	decentralized nodes	BitTorrent

rvices Questions

	Answer
es Architecture?	system composed of **small independent services** communicating through APIs
	bounded context and service independence
	scalability, independent deployment
	distributed complexity, debugging difficulty

uestions

Idea
arate model, view, controller
ices discover each other dynamically
lishers send events, subscribers receive them
ibute requests to multiple services and aggregate results
ibute requests across servers
ral controller manages workflow
ices coordinate through events
ributed transactions with compensation
ributed big data processing

(Very Common)

on	Choreography
	decentralized
orchestrator	event-driven
	high
	harder